SEASUG Paper 29-2026

SAS Macro Debugging Techniques for Mere Mortals

Kirk Paul Lafler, sasNerd

Abstract

SAS macro programming is powerful, but when things go wrong, debugging can feel complicated, frustrating, and time-consuming. This Hands-On Workshop is designed for SAS users who understand macro basics but struggle to diagnose and fix macro-related issues efficiently. Attendees will learn how to systematically debug macro programs by leveraging SAS system options, diagnostic tools, and structured troubleshooting approaches. This paper and session emphasizes real-world debugging scenarios, including resolving macro variable scope issues, identifying compilation vs. execution errors, handling quoting pitfalls, and interpreting cryptic log messages.

Through guided exercises, attendees will gain practical experience using tools such as MPRINT, MLOGIC, SYMBOLGEN, and OPTIONS SOURCE2, as well as techniques for isolating logic errors, validating input parameters, and testing macro-output incrementally. At the end of this paper and session, attendees will possess a repeatable debugging framework that transforms macro troubleshooting from guesswork into a disciplined, efficient process.

Introduction

SAS macro programming is one of the most powerful, and often most misunderstood, capabilities in the SAS ecosystem. It enables automation, dynamic code generation, and scalable analytics across complex workflows. Yet for many programmers, the moment a macro stops working as expected productivity grinds to a halt. The SAS Log fills with cryptic messages, variables fail to resolve, and what seemed like a simple task quickly becomes a frustrating puzzle.

The root of this challenge lies in how the SAS macro facility operates. Unlike standard DATA step or PROC code, macros do not execute data logic directly, they generate SAS code that is then compiled and executed. This additional layer of abstraction can obscure the source of errors, making it difficult to distinguish between issues in the macro logic itself and problems in the generated SAS code. As a result, even experienced SAS users can find macro debugging unintuitive and time-consuming.

This Hands-On Workshop (HOW) is designed to bridge that gap. Rather than treating debugging as an ad hoc activity, we approach it as a structured skill, one that can be learned, practiced, and mastered. Using the SASHELP.Heart dataset (5,209 observations and 17 variables), each concept is grounded with realistic analytical scenarios that mirror everyday SAS usage.

Attendees are introduced to the essential tools and techniques that bring clarity to macro execution, including SAS system debugging options, log interpretation strategies, and targeted diagnostic methods. More importantly, the workshop session emphasizes *how to think* when debugging macros, how to isolate issues, trace logic, validate assumptions, and systematically narrow down root causes.

Whether you are new to macros or have been using them for years, this session aims to replace uncertainty with confidence. By the end, you will not only understand why macro errors occur but also possess a practical toolkit to resolve them efficiently, turning one of SAS programming's most challenging areas into a manageable and even predictable process.

Dataset Used in Examples

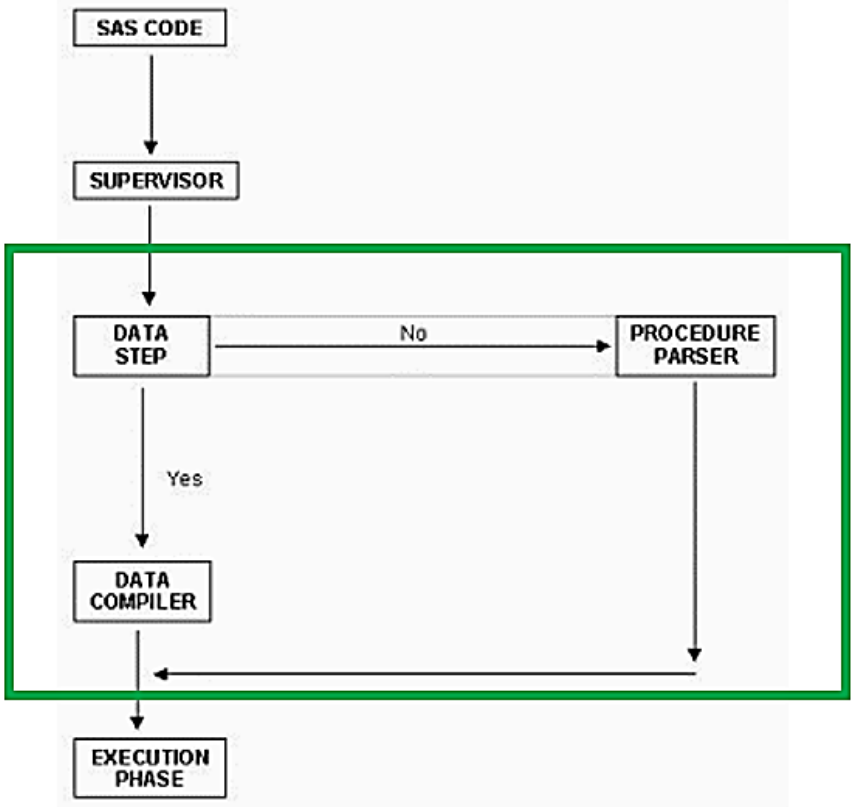
The code examples shown in this paper and Hands-On Workshop (HOW) use the Heart.sas7bdat SAS dataset found in the SASHELP library.

Number of Observations: 5,209
Number of Variables: 17

Status	DeathCause	AgeCHDdiag	Sex	AgeAtStart	Height	Weight	Diastolic	Systolic	MRW	Smoking	AgeAtDeath	Cholesterol	Chol_Status	BP_Status	Weight_Status	Smoking_Status
Dead	Other	.	Female	29	62.5	140	78	124	121	0	55	.		Normal	Overweight	Non-smoker
Dead	Cancer	.	Female	41	59.75	194	92	144	183	0	57	181	Desirable	High	Overweight	Non-smoker
Alive		.	Female	57	62.25	132	90	170	114	10	.	250	High	High	Overweight	Moderate (6-15)
Alive		.	Female	39	65.75	158	80	128	123	0	.	242	High	Normal	Overweight	Non-smoker
Alive		.	Male	42	66	156	76	110	116	20	.	281	High	Optimal	Overweight	Heavy (16-25)
Alive		.	Female	58	61.75	131	92	176	117	0	.	196	Desirable	High	Overweight	Non-smoker
Alive		.	Female	36	64.75	136	80	112	110	15	.	196	Desirable	Normal	Overweight	Moderate (6-15)
Dead	Other	.	Male	53	65.5	130	80	114	99	0	77	276	High	Normal	Normal	Non-smoker
Alive		.	Male	35	71	194	68	132	124	0	.	211	Borderline	Normal	Overweight	Non-smoker
Dead	Cerebral Vascul	.	Male	52	62.5	129	78	124	106	5	82	284	High	Normal	Normal	Light (1-5)
Alive		.	Male	39	66.25	179	76	128	133	30	.	225	Borderline	Normal	Overweight	Very Heavy (> 25)
Alive		57	Male	33	64.25	151	68	108	118	0	.	221	Borderline	Optimal	Overweight	Non-smoker
Alive		55	Male	33	70	174	90	142	114	0	.	188	Desirable	High	Overweight	Non-smoker
Alive		79	Male	57	67.25	165	76	128	118	15	.	.		Normal	Overweight	Moderate (6-15)
Alive		66	Male	44	69	155	90	130	105	30	.	292	High	High	Normal	Very Heavy (> 25)
Alive		.	Female	37	64.5	134	76	120	108	10	.	196	Desirable	Normal	Normal	Moderate (6-15)
Alive		.	Male	40	66.25	151	72	132	112	30	.	192	Desirable	Normal	Overweight	Very Heavy (> 25)
Dead	Cancer	56	Male	56	67.25	122	72	120	87	15	72	194	Desirable	Normal	Underweight	Moderate (6-15)
Alive		.	Female	42	67.75	162	96	138	119	1	.	200	Borderline	High	Overweight	Light (1-5)
Dead	Coronary Heart	74	Male	46	66.5	157	84	142	116	30	76	233	Borderline	High	Overweight	Very Heavy (> 25)
Alive		.	Female	37	66.25	148	78	110	112	15	.	192	Desirable	Optimal	Overweight	Moderate (6-15)
Alive		.	Female	45	64	147	74	120	119	5	.	209	Borderline	Normal	Overweight	Light (1-5)
Alive		.	Female	59	65.75	156	74	156	122	0	.	200	Borderline	High	Overweight	Non-smoker
Alive		.	Female	36	63.75	122	84	132	102	0	.	184	Desirable	Normal	Normal	Non-smoker
Alive		.	Female	50	67.5	185	88	150	136	15	.	228	Borderline	High	Overweight	Moderate (6-15)
Alive		.	Female	35	66	123	76	132	93	0	.	150	Desirable	Normal	Normal	Non-smoker
Alive		.	Male	42	72.25	182	78	136	113	0	.	221	Borderline	Normal	Overweight	Non-smoker

Table 1. Heart SAS Dataset found in the SASHELP Library.

SAS Processing Flow (without the SAS Macro Language)



The SAS processing flow is handled by the following system of internal components:

1. **Input Stack** – Your submitted SAS code is handed to the SAS Supervisor
2. **SAS Compiler**
   - ✓ Parses generated code
   - ✓ Checks syntax
3. **Execution Engine**
   - ✓ Runs DATA steps and PROC steps

The SAS program control flow without the SAS macro language consists of one or more DATA and/or PROC steps and is illustrated in the following image.

Introduction to the SAS Macro Language

There's a lot of confusion about the SAS macro language, so let's jump right into why it often feels so confusing. To really understand it, you need a clear mental model of what it is, where it lives, and what it does in the SAS system. The SAS macro language is a meta-language; a language that writes SAS code rather than executes data operations itself. At its core, the SAS macro language:

- ✓ Is a text substitution and code generation facility
- ✓ Operates before SAS compiles your program
- ✓ Enables dynamic, reusable, and parameter-driven code

The SAS macro language is NOT:

- ✓ A data manipulation language
- ✓ A replacement for DATA steps or PROC steps
- ✓ A tool that directly accesses datasets

Think of the SAS Macro Language as a program that writes other programs and exists to solve four major needs:

1. **Code Reusability** – where code is written once and run many times with different inputs.
2. **Automation** – generate repetitive code automatically.
3. **Parameterization** – create flexible programs driven by inputs.
4. **Dynamic Code Generation** – adapt code based on conditions, metadata, or environment.

Historically, SAS programs were:

- ✓ Long
- ✓ Repetitive
- ✓ Hard to maintain

The macro language was introduced to:

- ✓ Reduce duplication
- ✓ Improve maintainability
- ✓ Enable scalable program design

Macro Variables (or Text Containers)

Macro variables are central to the macro language and possess the following characteristics:

- ✓ Store **text strings**
- ✓ Referenced with &name
- ✓ Resolved before SAS execution

Features of DATA / PROC Steps versus Macro Language

There are important concepts that provide an essential distinction between the non-macro and macro world.

Feature	DATA / PROC Step Language	Macro Language
Timing	Execution-time	Compile-time
Purpose	Process data	Generate code
Syntax	DATA/PROC syntax	% and &
Data Access	Yes	No

A considerable amount of confusion disappears when you adopt the following premise:

- ✓ SAS Language = Run-Time Data Processor
- ✓ SAS Macro Language = Compile-Time Code Generator

A simple analogy can be applied to help better understand the difference between the world of DATA / PROC Step versus the Macro Language. Think of building a house:

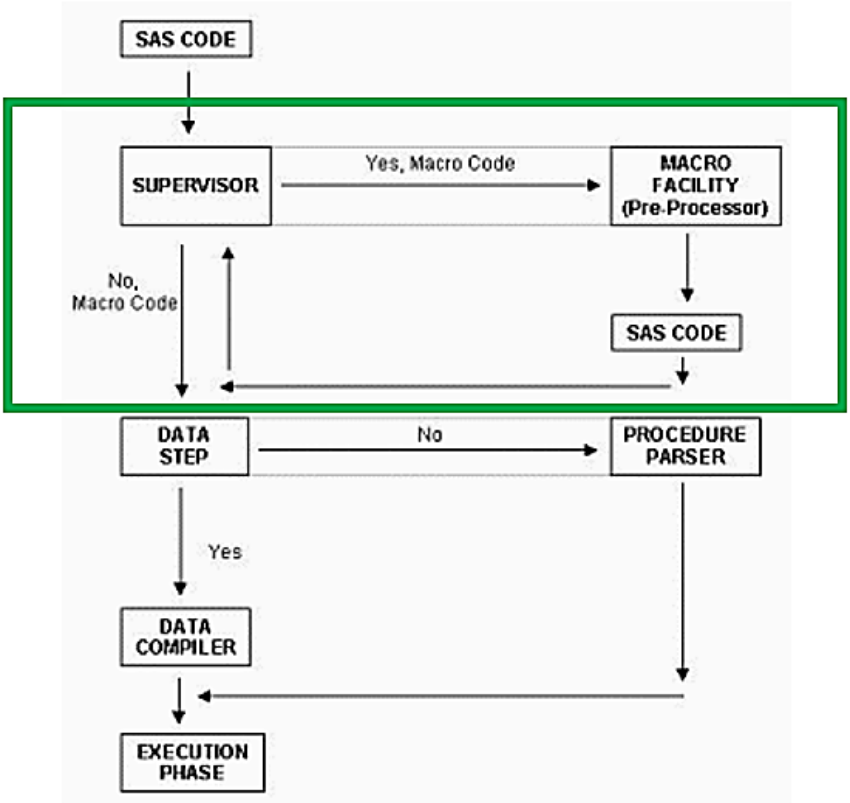
- ✓ **Macro language** = Architect drawing blueprints
- ✓ **SAS engine** = Construction crew building the house

The architect doesn't lay the bricks; the architect defines how the house should be built.

One final thought about the SAS macro language is that it is powerful because it allows you to:

- ✓ Build intelligent and flexible programs
- ✓ Eliminate code repetition
- ✓ Scale solutions across datasets, steps, and environments

Where the Macro Language Lives in SAS



The macro language is handled by a dedicated system of internal components. The typical SAS processing flow consists of the following components:

1. **Input Stack** – Your submitted SAS code is handed to the SAS Supervisor
2. **Macro Processor** – (Pre-processor – macro language lives here)
   - ✓ Resolves macro variables (&var)
   - ✓ Executes macro logic (%if, %do)
   - ✓ Expands macros into standard SAS code
3. **SAS Compiler**
   - ✓ Parses generated code
   - ✓ Checks syntax
4. **Execution Engine**
   - ✓ Runs DATA steps and PROC steps

The SAS program control flow consisting of one or more macro variables (&var) and macro logic (%if, %do, etc.); followed by one or more DATA and/or PROC steps is illustrated in the following image.

Introduction to Macro Debugging

Macro debugging in SAS is the practice of identifying, analyzing, and resolving issues that arise within the macro facility; the component responsible for generating dynamic SAS code. Unlike traditional DATA step or PROC step debugging, macro debugging requires thinking in terms of *code generation* and *text substitution*, not just code execution.

At its core, the SAS macro processor builds SAS statements *before* they are compiled and executed. This extra layer introduces both power and complexity: what you write is often not what SAS ultimately executes. As a result, successful macro debugging depends on gaining visibility into how your macros resolve, what code they generate, and how that generated code behaves.

Structured Macro Debugging Workflow

A disciplined approach to macro debugging dramatically improves efficiency because it replaces guesswork with a repeatable, evidence-driven process. In the macro environment; where issues can originate from text generation, macro logic, or the resulting SAS code; structure is what prevents wasted time chasing the wrong layer of the problem. The following steps illustrate a structured debugging process.

Step 1: Isolate the Problem

- ✓ Run small sections
- ✓ Test macro independently

Step 2: Turn on Debugging Options

```
options mprint mlogic symbolgen;
```

Step 3: Inspect the Log

- ✓ Look for unresolved variables
- ✓ Check generated code

Step 4: Validate Generated Code

- ✓ Copy from log
- ✓ Run outside macro

Step 5: Simplify

- ✓ Reduce macro complexity
- ✓ Test incrementally

Why Discipline Matters in Macro Debugging

A structured approach helps you avoid:

- ✓ Chasing errors in the wrong layer
- ✓ Ignoring unresolved macro variables
- ✓ Misreading log messages
- ✓ Debugging entire programs instead of isolated components
- ✓ Overlooking simple syntax issues in generated code

Without a structured approach, macro debugging often becomes:

- ✓ Trial-and-error driven
- ✓ Log-overwhelming and confusing
- ✓ Time-consuming due to misdiagnosed issues

A disciplined workflow ensures that you:

- ✓ Identify *where* the problem originates (macro vs generated code)
- ✓ Reduce noise in the log
- ✓ Focus on root causes faster
- ✓ Avoid introducing new errors while fixing existing ones

Core Principles of a Disciplined Debugging Approach

To debug SAS macros effectively and consistently, it is essential to follow a set of core principles that bring structure, clarity, and precision to the debugging process.

Hands-On Exercise #1

Controlling and Enabling the Debugging Environment with SAS System Options

SAS provides powerful system options that allow you to control the level of visibility into macro execution. Instead of guessing what your macro is doing, you can instruct SAS to reveal how macro variables are resolved, how logic flows, and what code is ultimately generated. The key is to enable only what you need, when you need it, to avoid overwhelming the log with unnecessary detail.

Objective

The goal of this exercise is to:

- ✓ Understand what each debugging option does
- ✓ Learn how to interpret the SAS log output
- ✓ Observe how macros resolve into executable SAS code

Task #1: Identify how &var resolves in the log.

The most used debugging options are:

```
options mprint mlogic symbolgen;
```

What Each Option Does

1. MPRINT – View Generated SAS Code

- ✓ Displays the actual SAS statements produced by the macro
- ✓ Helps to verify whether the generated code is syntactically correct

2. MLOGIC – Trace Macro Logic Flow

- ✓ Shows entry and exit of macros
- ✓ Displays %IF/%THEN evaluations and loop execution

3. SYMBOLGEN – Resolve Macro Variables

- ✓ Prints the values of macro variables as they are resolved
- ✓ Useful for detecting missing or incorrect values

Code

```
options mprint mlogic symbolgen;

%let var=weight;
proc means data=SASHELP.Heart;
  var &var;
run;
```

But discipline also means:

- ✓ Turning options off when done
- ✓ Avoiding excessive log clutter in large programs

Interpreting the Log Output

You will now see detailed log entries such as:

```
SYMBOLGEN: Macro variable VAR resolves to Weight
MLOGIC: Beginning execution.
MPRINT: proc means data=SASHELP.Heart;
MPRINT: var weight;
MPRINT: run;
MLOGIC: Ending execution.
```

What This Tells You

- ✓ SYMBOLGEN confirms macro variable values
- ✓ MLOGIC shows decision-making and flow
- ✓ MPRINT reveals the exact SAS code executed

Hands-On Exercise #2

Misspelled Macro Variable Debugging

Misspelled macro variables are one of the most common; and often overlooked; sources of error in SAS macro programming. Because the macro processor performs text substitution, it does not inherently “know” whether a variable name is correct. Instead, it attempts to resolve what you provide. If the name is incorrect, SAS typically does not stop immediately, which can make these errors subtle and harder to detect.

Objective

This exercise will help you:

- ✓ Identify unresolved macro variables in the SAS log
- ✓ Understand how SAS behaves when a macro variable is misspelled
- ✓ Use debugging options to pinpoint the issue quickly
- ✓ Apply best practices to prevent similar errors

Code

```
%macro summary;
    %let variable=AgeAtStart;
    proc means data=SASHELP.Heart;
        var &varible;
    run;
%mend;

%summary;
```

What's the Issue?

- ✓ Macro variable defined: variable
- ✓ Macro variable used: varible (**misspelled**)

Because of this mismatch, the macro processor cannot resolve &varible.

Task #2: Use SYMBOLGEN to diagnose the issue.

Step 1: Run Without Debugging Options

When executed, the SAS log will show a warning like:

WARNING: Apparent symbolic reference VARIBLE not resolved.

What This Means

- ✓ SAS could not find a macro variable named VARIBLE
- ✓ Instead of stopping, SAS passes the unresolved text into the generated code

Resulting Generated Code (Implicit)

```
proc means data=SASHELP.Heart;
    var &varible;
run;
```

This leads to a **secondary error** during execution because **&varible** is not a valid variable name.

Step 2: Enable Debugging Options

Turn on macro debugging:

```
options mprint mlogic symbolgen;
```

Re-run the macro:

```
%summary;
```

Enhanced Log Output

```
SYMBOLGEN: Macro variable VARIABLE resolves to AgeAtStart
WARNING: Apparent symbolic reference VARIBLE not resolved.
MPRINT(SUMMARY): proc means data=SASHELP.Heart;
MPRINT(SUMMARY): var &varible;
MPRINT(SUMMARY): run;
```

How to Interpret This

- ✓ SYMBOLGEN confirms that VARIABLE is correctly assigned
- ✓ The warning highlights that VARIBLE does not exist
- ✓ MPRINT shows the invalid generated code

Key Insight

Even though the macro variable was defined correctly, it was never used correctly, leading to failure downstream.

Step 3: Fix the Issue

Correct the spelling:

```
%macro summary;
    %let variable=AgeAtStart;
    proc means data=SASHELP.Heart;
        var &variable;
    run;
%mend;

%summary;
```

Expected Result

- ✓ No warnings about unresolved macro variables
- ✓ PROC MEANS executes successfully
- ✓ Summary statistics for AgeAtStart are displayed

Why This Error Is Dangerous

Misspelled macro variables can:

- ✓ Silently propagate into generated code
- ✓ Produce misleading or incomplete results
- ✓ Be difficult to trace in large programs

Unlike syntax errors, they often do **not immediately stop execution**, making them particularly risky in production environments.

Best Practices to Prevent This Issue

1. Use SYMBOLGEN Regularly

It immediately reveals whether variables are resolved correctly.

2. Adopt Consistent Naming Conventions

- ✓ Use meaningful, standardized names
- ✓ Avoid similar-looking variable names (variable vs varible)

3. Use %PUT for Validation

```
%put NOTE: variable = &variable;
```

4. Enable SAS Options for Error Detection

```
options merror serror;
```

- ✓ MERROR → warns about undefined macros
- ✓ SERROR → warns about unresolved macro variables

5. Validate Before Use

```
%if %symexist(variable) = 0 %then %do;
    %put ERROR: Macro variable VARIABLE does not exist.;
%end;
```

Key Takeaways

- ✓ SAS does not automatically fail on unresolved macro variables
- ✓ Warnings in the log are critical; never ignore them
- ✓ SYMBOLGEN is your first line of defense
- ✓ Misspellings can propagate into valid looking but incorrect code
- ✓ Small typos can lead to large downstream issues

Hands-On Exercise #3

Macro Variable Scope Debugging

Macro variable scope is a frequent source of confusion; and bugs; in SAS macro programming. Even when your logic is correct, improper handling of scope can lead to unexpected values, overwritten variables, or missing results. This exercise demonstrates how scope works and how to explicitly control it using %LOCAL and %GLOBAL.

Objective

By completing this exercise, you will:

- ✓ Understand the difference between **local** and **global** macro variables
- ✓ Recognize how nested macros inherit (or fail to inherit) variables
- ✓ Learn how to explicitly control scope using %LOCAL and %GLOBAL
- ✓ Prevent unintended side effects in larger macro programs

Code

```
%macro outer;
    %let var=Systolic;

    %macro inner;
        proc means data=SASHELP.Heart;
            var &var;
        run;
    %mend;

    %inner;
%mend;

%outer;
```

What's Happening Here?

At first glance, this code appears to work correctly; and in many cases, it *will*. However, it relies on implicit scope behavior, which can become problematic in more complex programs.

Key Points

- ✓ %let var=age; is created inside outer
- ✓ By default, this creates a **local macro variable** within outer
- ✓ The inner macro can access var because it is nested and executed within outer

Why This Is Risky

- ✓ If inner is called outside outer, &var will not resolve
- ✓ If another macro defines var, it may overwrite or conflict
- ✓ Debugging becomes difficult when scope is implicit rather than explicit

Task #3: Specify the proper scope using %GLOBAL or %LOCAL.

Step 1: Observe Scope Behavior

Enable debugging:

```
options symbolgen;
```

Add a diagnostic statement:

```
%put NOTE: var = &var;
```

You may notice:

- ✓ var resolves correctly *inside* outer
- ✓ But may not exist outside it

Solution 1: Use %LOCAL (Best Practice for Encapsulation)

Explicitly declare var as local to the outer macro:

```
%macro outer;
    %local var;
    %let var=Systolic;

    %macro inner;
        proc means data=SASHELP.Heart;
            var &var;
        run;
    %mend;

    %inner;
%mend;

%outer;
```

Why This Is Better

- ✓ Makes scope **explicit and predictable**
- ✓ Prevents accidental overwriting of global variables
- ✓ Keeps variables contained within the macro

Solution 2: Use %GLOBAL (When Sharing Across Macros Is Required)

If you want var to be accessible outside outer:

```
%macro outer;
    %global var;
    %let var=Systolic;

    %macro inner;
        proc means data=SASHELP.Heart;
            var &var;
        run;
    %mend;

    %inner;
%mend;

%outer;
```

When to Use %GLOBAL

- ✓ When multiple macros need to share the same variable
- ✓ When values must persist beyond a single macro execution

Caution

Global variables:

- ✓ Persist across the entire SAS session
- ✓ Can be overwritten unintentionally
- ✓ Make debugging more complex in large systems

Step 2: Demonstrate the Difference

Test Global Scope

```
%outer;
%put NOTE: Outside macro, var = &var;
```

- With %GLOBAL → works
- With %LOCAL → produces warning:

```
WARNING: Apparent symbolic reference VAR not resolved.
```

Best Practices for Macro Variable Scope

1. Default to %LOCAL

- ✓ Safer and more modular
- ✓ Reduces unintended side effects

2. Use %GLOBAL Sparingly

- ✓ Only when sharing is required
- ✓ Document clearly when used

3. Declare Variables Explicitly
Avoid relying on implicit scope creation.

4. Debug Scope with Diagnostic Tools
%put _local_;
%put _global_;

5. Avoid Reusing Generic Names
Names like var, data, and temp increase collision risk.

Hands-On Exercise #4

Debugging Conditional Logic

Conditional logic is central to macro programming, but it is also a frequent source of bugs. Because %IF/%THEN statements are evaluated during macro processing (before execution), errors in logic often do not produce obvious failures; they simply result in code not being generated. This makes it essential to trace the logic flow explicitly when debugging.

Objective
In this exercise, you will:

- ✓ Trace how macro conditional logic is evaluated
- ✓ Add diagnostic output to understand decision-making
- ✓ Identify why expected code may not be executed
- ✓ Apply best practices for debugging %IF/%THEN logic

Code

```
%macro check_gender(gender=);
    %if &gender = M %then %do;
        proc freq data=SASHELP.Heart;
            tables sex;
        run;
    %end;
%mend;

%check_gender(gender=F);
```

What's the Issue?

- ✓ The macro checks whether gender = M
- ✓ The macro is called with gender=F
- ✓ Therefore, the condition evaluates to FALSE
- ✓ As a result, no code is generated

Key Insight
There are no errors; but also, no output. This is a classic macro debugging scenario where:
The program runs, but nothing happens.

Task #4: Add debugging output to trace logic flow.

Step 1: Add Debugging Output with %PUT

To trace the logic, instrument the macro:

```
%macro check_gender(gender=);

    %put NOTE: Entering CHECK_GENDER macro;
    %put NOTE: gender parameter = &gender;

    %if &gender = M %then %do;
        %put NOTE: Condition met (gender = M). Running PROC FREQ;

        proc freq data=SASHELP.Heart;
            tables sex;
        run;
    %end;
    %else %do;
        %put NOTE: Condition NOT met (gender ne M). No action taken;
    %end;

    %put NOTE: Exiting CHECK_GENDER macro;

%mend;

%check_gender(gender=F);
```

Expected Log Output

```
NOTE: Entering CHECK_GENDER macro
NOTE: gender parameter = F
NOTE: Condition NOT met (gender ne M). No action taken
NOTE: Exiting CHECK_GENDER macro
```

What This Reveals

- ✓ The macro is executing correctly
- ✓ The parameter is being passed properly
- ✓ The condition evaluates to FALSE
- ✓ The lack of output is intentional; not an error

Step 2: Enable System Debugging Options

For deeper insight, enable:

```
options mlogic symbolgen;
```

Additional Log Details

```
SYMBOLGEN: Macro variable GENDER resolves to F
MLOGIC(CHECK_GENDER): %IF condition (&gender = M) is FALSE
```

Step 3: Improve Robustness of Conditional Logic

Common Pitfall: Case Sensitivity

If a user enters lowercase:

```
%check_gender(gender=m);
```

The condition fails because:

- "m" ≠ "M"

Solution: Normalize Input

```
%macro check_gender(gender=);

    %let gender = %upcase(&gender);

    %put NOTE: Normalized gender = &gender;

    %if &gender = M %then %do;
        %put NOTE: Condition met (gender = M). Running PROC FREQ;

        proc freq data=SASHELP.Heart;
            tables sex;
        run;
    %end;
    %else %do;
        %put NOTE: Condition NOT met. No action taken;
    %end;

%mend;
```

Step 4: Handle Missing or Invalid Input

```
%macro check_gender(gender=);

    %if %length(&gender)=0 %then %do;
        %put ERROR: Gender parameter is missing.;
        %return;
    %end;

    %let gender = %upcase(&gender);

    %if &gender = M %then %do;
        proc freq data=SASHELP.Heart;
            tables sex;
        run;
    %end;
    %else %do;
        %put WARNING: Gender value &gender not recognized or not processed.;
    %end;

%mend;
```

Hands-On Exercise #5

Loop Debugging

Loops are a powerful feature of the SAS macro language, allowing you to generate repetitive code dynamically. However, when loops do not behave as expected; running too many times, too few times, or not at all; they can be difficult to debug without proper visibility into the macro processor. This exercise demonstrates how to trace loop execution using **MLOGIC** and enhance visibility with additional debugging techniques.

Objective

By completing this exercise, you will:

- ✓ Understand how %DO loops execute in the macro processor
- ✓ Use MLOGIC to trace loop iteration behavior
- ✓ Identify common loop-related issues
- ✓ Apply debugging techniques to validate loop control variables

Code

```
%macro loop_test;
    %do i=1 %to 3;
        %put Iteration: &i;
    %end;
%mend;

%loop_test;
```

Task #5: Use MLOGIC to trace loop execution.

What This Macro Does

- ✓ Initializes a loop from i = 1 to 3
- ✓ Executes %PUT during each iteration
- ✓ Writes iteration values to the SAS log

Expected Output

Iteration: 1
Iteration: 2
Iteration: 3

Step 1: Enable MLOGIC for Loop Tracing

Turn on macro logic tracing:

```
options mlogic;
```

Run the macro again:

```
%loop_test;
```

Enhanced Log Output

```
MLOGIC(LOOP_TEST): Beginning execution.
MLOGIC(LOOP_TEST): %DO loop beginning; index variable I; start=1; stop=3; by=1.
MLOGIC(LOOP_TEST): %DO loop index variable I is now 1
Iteration: 1
MLOGIC(LOOP_TEST): %DO loop index variable I is now 2
Iteration: 2
MLOGIC(LOOP_TEST): %DO loop index variable I is now 3
Iteration: 3
MLOGIC(LOOP_TEST): %DO loop ending; index variable I has value 4.
MLOGIC(LOOP_TEST): Ending execution.
```

How to Interpret This

- ✓ MLOGIC shows:
  - Loop initialization (start, stop, by)
  - Each iteration value of i
  - When the loop ends
- ✓ The final value (i = 4) indicates the loop termination condition was reached

Key Insight

You can now see exactly how many times the loop runs and why.

Step 2: Add Custom Debugging Output

Enhance traceability with %PUT:

```
%macro loop_test;
    %put NOTE: Starting loop;

    %do i=1 %to 3;
        %put NOTE: Iteration = &i;
    %end;

    %put NOTE: Loop complete;
%mend;

%loop_test;
```

Step 3: Debug a Common Loop Issue

Problem: Loop Does Not Execute

```
%macro loop_test;
    %do i=5 %to 3;
        %put Iteration: &i;
    %end;
%mend;

%loop_test;
```

Result

- ✓ No output is produced

With MLOGIC Enabled

```
MLOGIC(LOOP_TEST): %DO loop beginning; index variable I; start=5; stop=3; by=1.
MLOGIC(LOOP_TEST): Loop will not be executed.
```

Insight

The start value is greater than the stop value, so the loop never runs.

Step 4: Debug Incorrect Increment (BY Value)

```
%macro loop_test;
    %do i=1 %to 5 %by 2;
        %put Iteration: &i;
    %end;
%mend;

%loop_test;
```

Output

Iteration: 1

Iteration: 3

Iteration: 5

Debugging Insight

Use MLOGIC to confirm step increments and ensure expected behavior.

Step 5: Debug Infinite or Unexpected Loops

Macro loops can behave unexpectedly if boundaries are dynamic:

```
%macro loop_test(n);
    %do i=1 %to &n;
        %put Iteration: &i;
    %end;
%mend;

%loop_test(0);
```

Result

- ✓ No iterations occur

Best Practice

Validate loop boundaries:

```
%if &n <= 0 %then %do;
    %put WARNING: Loop will not execute because n=&n;
%end;
```

Hands-On Exercise #6

Macro Quoting Debugging

Macro quoting is one of the more subtle; and often misunderstood; areas of SAS macro debugging. Problems arise when special characters (such as apostrophes, commas, parentheses, or semicolons) are interpreted by the macro processor as syntax rather than data. When this happens, your macro may fail, behave unpredictably, or produce incomplete results. This exercise focuses on diagnosing and fixing a quoting issue caused by an apostrophe in the input value.

Objective

By completing this exercise, you will:

- ✓ Understand why special characters break macro processing
- ✓ Learn how macro quoting functions protect text
- ✓ Apply %STR, %NRSTR, %BQUOTE, and %SUPERQ appropriately
- ✓ Fix real-world quoting issues safely and reliably

Quoting Issue

```
%macro test_quote(val=);
    %put Value is &val;
%mend;

%test_quote(val=John's Data);
```

Task #6: Fix using macro quoting functions.

What's the Issue?

The value passed:

John's Data

contains an **apostrophe (')**, which SAS may interpret as:

- ✓ The start or end of a string literal
- ✓ A delimiter that breaks macro parsing

Typical Log Symptoms

You may see:

```
ERROR: More positional parameters found than defined.
```

or

```
WARNING: Apparent symbolic reference not resolved.
```

Root Cause

The macro processor is trying to interpret 's Data as part of macro syntax rather than a literal string.

Step 1: Fix Using %STR (Compile-Time Masking)

Use %STR to mask special characters when passing the parameter:

```
%test_quote(val=%str(John's Data));
```

Why This Works

- ✓ %STR masks special characters at **compile time**
- ✓ The macro processor treats the entire string as literal text

Step 2: Use %NRSTR (Masking with Macro Triggers)

If your string contains macro triggers like & or %, use %NRSTR:

```
%test_quote(val=%nrstr(John's Data));
```

Difference

- ✓ %STR → masks special characters
- ✓ %NRSTR → masks special characters **and macro triggers**

Step 3: Handle Runtime Resolution with %BQUOTE

If the value is already stored in a macro variable:

```
%let name=John's Data;

%macro test_quote(val=);
  %put Value is %bquote(&val);
%mend;

%test_quote(val=&name);
```

Why Use %BQUOTE

- ✓ Masks special characters **at execution time**
- ✓ Useful when values are dynamically generated

Step 4: Use %SUPERQ for Maximum Safety

The safest and most robust approach:

```
%let name=John's Data;

%macro test_quote(val=);
  %put Value is %superq(val);
%mend;

%test_quote(val=&name);
```

Why %SUPERQ Is Powerful

- ✓ Prevents **any further resolution**
- ✓ Safely retrieves macro variable values without triggering parsing
- ✓ Ideal for debugging and logging

Comparison of Quoting Functions

Function	When to Use	Key Feature
%STR	Hardcoded text	Compile-time masking
%NRSTR	Text with & or %	Masks macro triggers
%BQUOTE	Resolved values	Runtime masking
%SUPERQ	Macro variables	Prevents re-resolution

Step 5: Add Debugging Visibility

Enable debugging options:

```
options symbolgen;
```

Add diagnostic output:

```
%put NOTE: Raw value = &val;
%put NOTE: Quoted value = %superq(val);
```

Hands-On Exercise #7

Debugging Generated Code

One of the most powerful techniques in SAS macro debugging is examining the actual SAS code generated by a macro. Because macros produce code dynamically, errors often originate not in the macro logic itself, but in the code it generates. The MPRINT system option allows you to see this generated code directly in the SAS log; turning the macro processor from a “black box” into a transparent system.

Objective

By completing this exercise, you will:

- ✓ Use MPRINT to view the SAS generated code
- ✓ Identify discrepancies between intended and actual output
- ✓ Debug macro naming and logic issues
- ✓ Validate correctness of dynamically generated code

Code

```
%macro report_data(ds=SASHELP.Heart,type=Detail);
  %if &type=Detail %then %do;
      proc print data=&ds;
      run;
  %end;
  %else %if &type=Summary %then %do;
      proc means data=&ds min max range mean median mode std variance;
        class sex;
      run;
  %end;
%mend report_data;

%report_data(type=Summary);
```

Task #7: Examine generated SAS code in the log.

Step 1: Enable MPRINT

To begin debugging, enable the MPRINT option:

```
options mprint;
```

Now re-run the macro call:

```
%report_data(type=Summary);
```

Step 2: Examine the SAS Log

With MPRINT enabled, the log will include the generated SAS code:

```
MPRINT(REPORT_DATA): proc means data=SASHELP.Heart min max range mean median mode std
variance;
MPRINT(REPORT_DATA): class sex;
MPRINT(REPORT_DATA): run;
```

What This Tells You

- ✓ The macro correctly evaluated the %IF/%ELSE logic
- ✓ The Summary branch was selected
- ✓ The default dataset (SASHELP.Heart) was used
- ✓ The generated code is syntactically valid

Key Insight

You are no longer interpreting macro logic; you are inspecting actual executable SAS code.

Step 3: Compare Intended vs. Generated Code

Ask yourself:

- ✓ Is this the code I expected?
- ✓ Are all parameters resolving correctly?
- ✓ Is anything missing (e.g., semicolons, statements)?

Even small discrepancies can lead to:

- ✓ Syntax errors
- ✓ Incorrect results
- ✓ Silent failures

Step 4: Test Alternate Execution Path

Run the macro with a different parameter:

```
%report_data(type=Detail);
```

Log Output

```
MPRINT(REPORT_DATA): proc print data=SASHELP.Heart;
MPRINT(REPORT_DATA): run;
```

This confirms:

- ✓ Conditional branching works correctly
- ✓ Both logic paths generate valid code

Step 5: Debug a Common Issue – Case Sensitivity

```
%report_data(type=summary);
```

Problem

- ✓ "summary" does not match "Summary"
- ✓ Condition evaluates to FALSE
- ✓ No code is generated

Log Behavior

```
NOTE: No MPRINT output generated.
```

Solution: Normalize Input

```
%macro report_data(ds=SASHELP.Heart,type=Detail);

  %let type = %upcase(&type);

  %if &type=DETAIL %then %do;
      proc print data=&ds;
      run;
```

```
  %end;
  %else %if &type=SUMMARY %then %do;
      proc means data=&ds min max range mean median mode std variance;
        class sex;
      run;
  %end;
  %else %do;
      %put WARNING: Invalid TYPE value: &type;
  %end;

%mend report_data;
```

Step 6: Validate Generated Code Independently

A highly effective debugging technique:

1. Copy the MPRINT output from the log
2. Paste it into a new SAS program
3. Run it independently

Example

```
proc means data=SASHELP.Heart min max range mean median mode std variance;
  class sex;
run;
```

Interpretation

- ✓ If this fails → issue is in generated code
- ✓ If this works → issue is in macro logic

Step 7: Combine MPRINT with Other Debugging Options

For full visibility:

```
options mprint mlogic symbolgen;
```

This gives you:

- ✓ MPRINT → generated code
- ✓ MLOGIC → decision flow
- ✓ SYMBOLGEN → variable resolution

Common Issues Revealed by MPRINT

1. Missing Semicolons

```
proc print data=&ds   /* Missing semicolon */
run;
```

2. Incorrect Parameter Resolution

```
proc means data=;
```

3. Invalid or Missing Dataset Names

```
proc print data=unknown;
```

4. Incomplete Code from Conditional Logic

Branches that fail silently due to unmet conditions.

Hands-On Exercise #8

Debugging a Missing Parameter

Missing parameters are a common source of macro failures in SAS. Because macro variables resolve through simple text substitution, SAS does not automatically enforce required parameters. If a parameter is omitted, the macro may still execute; but generate invalid or incomplete SAS code, often leading to confusing downstream errors. This exercise focuses on detecting missing parameters early and implementing defensive programming techniques to prevent execution when required inputs are not provided.

Objective

By completing this exercise, you will:

- ✓ Identify issues caused by missing macro parameters
- ✓ Understand how unresolved parameters affect generated code
- ✓ Implement validation checks using %IF and %LENGTH
- ✓ Add meaningful error messages and safe exits
- ✓ Build more robust and production-ready macros

Code

```
%macro analyze(ds=);
   proc means data=&ds;
   run;
%mend;

%analyze( );
```

Task #8: Add parameter validation and error handling.

What's the Issue?

- ✓ The macro parameter ds is not provided
- ✓ &ds resolves to a blank value

Generated Code (Implicit)

```
proc means data=;
run;
```

Log Output

```
ERROR: File WORK.DATA.DATA does not exist.
```

Why This Happens

- ✓ data= is empty → SAS interprets it incorrectly
- ✓ The error message does not clearly indicate that the parameter is missing
- ✓ Debugging becomes indirect and time-consuming

Step 1: Add Basic Parameter Validation

Use %LENGTH to check if the parameter is missing:

```
%macro analyze(ds=);
    %if %length(&ds)=0 %then %do;
        %put ERROR: The DS parameter is required but was not provided.;
        %return;
    %end;
    proc means data=&ds;
    run;
%mend;

%analyze( );
```

Improved Log Output

```
ERROR: The DS parameter is required but was not provided.
```

Key Improvement

- ✓ The error is now clear, immediate, and actionable
- ✓ The macro exits before generating invalid code

Step 2: Add Debugging Visibility

Enhance traceability with %PUT:

```
%macro analyze(ds=);

    %put NOTE: Starting ANALYZE macro;
    %put NOTE: DS parameter = &ds;

    %if %length(&ds)=0 %then %do;
        %put ERROR: Missing required parameter DS.;
        %return;
    %end;

    proc means data=&ds;
```

```
    run;

    %put NOTE: ANALYZE macro completed;

%mend;
```

Step 3: Validate Dataset Existence

Even if a parameter is provided, it may still be invalid.

```
%macro analyze(ds=);

    %if %length(&ds)=0 %then %do;
        %put ERROR: Missing required parameter DS.;
        %return;
    %end;

    %if %sysfunc(exist(&ds))=0 %then %do;
        %put ERROR: Dataset &ds does not exist.;
        %return;
    %end;

    proc means data=&ds;
    run;

%mend;

%analyze(ds=SASHELP.Heart);
```

Why This Matters

- ✓ Prevents execution on non-existent datasets
- ✓ Avoids misleading SAS errors
- ✓ Adds robustness to your macro

Step 4: Combine with Debugging Options

```
options mprint symbolgen;
```

Log Insight

SYMBOLGEN: Macro variable DS resolves to

This confirms the parameter is empty; reinforcing the need for validation.

Step 5: Add Default Values (Optional Strategy)

In some cases, you may want to provide a default:

```
%macro analyze(ds=SASHELP.Heart);

  proc means data=&ds;
  run;

%mend;

%analyze( );
```

Hands-On Exercise #9

Using %PUT for Debugging

The %PUT statement is one of the simplest yet most powerful tools for debugging SAS macros. It allows you to write custom messages to the SAS log, making it easy to trace macro execution, inspect variable values, and understand how calculations are performed step by step. Unlike system options (MPRINT, MLOGIC, SYMBOLGEN), %PUT gives you control over what information is displayed, enabling targeted and meaningful debugging output.

Objective

By completing this exercise, you will:

- ✓ Use %PUT to display macro variable values
- ✓ Trace intermediate steps in macro calculations
- ✓ Improve visibility into macro execution flow
- ✓ Apply structured logging techniques for debugging

Code

```
%macro calc;
  %let x=10;
  %let y=20;
  %let z=%eval(&x + &y);
  %put Result: &z;
%mend;

%calc;
```

Task #9: Enhance logging to show intermediate values.

What's Happening?

- ✓ x = 10
- ✓ y = 20
- ✓ z = x + y = 30
- ✓ %PUT prints the result

Output

Result: 30

Limitation

- ✓ Only the result is shown
- ✓ No visibility into intermediate steps

Task: Enhance Logging to Show Intermediate Values

To improve debugging, we will log each step of the calculation.

Step 1: Add Basic Debugging Statements

```
%macro calc;

  %put NOTE: Starting CALC macro;

  %let x=10;
  %put NOTE: Value of x = &x;

  %let y=20;
  %put NOTE: Value of y = &y;

  %let z=%eval(&x + &y);
  %put NOTE: Calculated z = &z;

  %put NOTE: Final Result = &z;

  %put NOTE: Ending CALC macro;

%mend;

%calc;
```

Enhanced Log Output

```
NOTE: Starting CALC macro
NOTE: Value of x = 10
NOTE: Value of y = 20
NOTE: Calculated z = 30
NOTE: Final Result = 30
NOTE: Ending CALC macro
```

What This Improves

- Full visibility into each step
- Easier identification of incorrect values
- Clear execution flow

Step 2: Use Automatic Variable Display

SAS provides a shortcut to display variable names and values:

```
%put &=x &=y &=z;
```

Output

```
X=10 Y=20 Z=30
```

Step 3: Add Expression-Level Debugging

Show the actual calculation:

```
%put NOTE: Evaluating expression: &x + &y;
```

Output

```
NOTE: Evaluating expression: 10 + 20
```

Step 4: Combine with Conditional Debugging

```
%if &z > 25 %then %put NOTE: z is greater than 25;
%else %put NOTE: z is 25 or less;
```

Step 5: Debug with %PUT *LOCAL* and *GLOBAL*

Display all macro variables in scope:

```
%put _local_;
%put _global_;
```

Benefit

- ✓ Reveals all variables and their values
- ✓ Helps identify scope-related issues

Step 6: Introduce an Error Scenario

```
%macro calc;
  %let x=10;
  %let y=abc;   /* Invalid numeric value */
  %let z=%eval(&x + &y);
  %put Result: &z;
%mend;
%calc;
```

Log Output

```
ERROR: A character operand was found in the %EVAL function...
```

Fix with Debugging

```
%put NOTE: x = &x;
%put NOTE: y = &y;
```

Insight

- ✓ Quickly reveals that y=abc is invalid for numeric calculation

Step 7: Advanced Logging Pattern

Create structured debug messages:

```
%put NOTE: [CALC] Step 1 - Initialize variables;
%put NOTE: [CALC] Step 2 - Compute z = &x + &y;
%put NOTE: [CALC] Step 3 - Result = &z;
```

Hands-On Exercise #10

For deeper debugging:
When working with complex SAS macro systems, macros often call other macros, sometimes multiple layers deep. In these situations, standard debugging options like MLOGIC and MPRINT may not provide enough clarity because they do not fully illustrate the hierarchy and depth of nested macro execution. To address this, SAS provides enhanced tracing options:

```
options mlogicnest mprintnest;
```

These options extend the capabilities of MLOGIC and MPRINT by adding nesting-level visibility, which is critical for understanding how macros interact in layered or modular designs.

What These Options Do

1. MLOGICNEST – Nested Macro Logic Tracing
- ✓ Builds on MLOGIC
- ✓ Displays macro execution flow with nesting levels
- ✓ Shows which macro is calling which, and in what order

2. MPRINTNEST – Nested Code Generation Tracing
- ✓ Builds on MPRINT
- ✓ Displays generated SAS code with indentation or labels indicating macro depth
- ✓ Allows you to see which macro generated which portion of code

Why This Matters

In simple macros, execution is linear and easy to follow. But in real-world systems:
- ✓ Macros call other macros
- ✓ Parameters are passed across layers
- ✓ Variables may be redefined or overridden
- ✓ Logic branches across multiple macro levels

Without nesting visibility, the log can become:
- ✓ Difficult to interpret
- ✓ Ambiguous about where code originates
- ✓ Time-consuming to debug

Example: Nested Macros Without Nesting Options

```
%macro inner;
  %put Inside INNER macro;
%mend;

%macro outer;
  %put Inside OUTER macro;
  %inner;
%mend;

options mlogic mprint;
%outer;
```

Log Output (Simplified)

```
MLOGIC(OUTER): Beginning execution.
Inside OUTER macro
MLOGIC(INNER): Beginning execution.
Inside INNER macro
MLOGIC(INNER): Ending execution.
MLOGIC(OUTER): Ending execution.
```

Limitation

- ✓ You see execution, but not clearly the **hierarchical relationship**

Example: With MLOGICNEST and MPRINTNEST

```
options mlogic mlogicnest mprint mprintnest;
%outer;
```

Enhanced Log Output

```
MLOGIC(OUTER): Beginning execution.
  MLOGIC(OUTER): %PUT Inside OUTER macro;
  Inside OUTER macro

  MLOGIC(INNER): Beginning execution.
    MLOGIC(INNER): %PUT Inside INNER macro;
    Inside INNER macro
  MLOGIC(INNER): Ending execution.

MLOGIC(OUTER): Ending execution.
```

What's Improved

- ✓ Clear **indentation** or nesting indicators
- ✓ Easy identification of:
  - Parent macro
  - Child macro
  - Execution order
- ✓ Better understanding of how control flows between macros

Real-World Scenario

Consider a reporting framework:

```
%macro generate_report;
    %prepare_data;
    %analyze_data;
    %print_results;
%mend;
```

Each of these macros may:

- ✓ Call additional macros
- ✓ Perform conditional logic
- ✓ Generate different code paths

With MLOGICNEST and MPRINTNEST, you can:

- ✓ Trace the full execution tree
- ✓ Identify exactly where errors originate
- ✓ Understand how data and parameters flow through the system

Common Issues These Options Help Diagnose

1. Incorrect Macro Call Order

- ✓ Identify whether macros are executing in the intended sequence

2. Parameter Passing Errors

- ✓ Track how parameters change across nested calls

3. Variable Scope Confusion

- ✓ Understand where variables are created and modified

4. Unexpected Code Generation

- ✓ See which macro generated problematic SAS code

5. Infinite or Recursive Macro Calls

- ✓ Detect repeated nested calls more easily

Key Takeaways

- ✓ Macro debugging becomes far more efficient and transparent when you enable key system options like MPRINT, MLOGIC, and SYMBOLGEN, giving you clear visibility into code execution and variable resolution
- ✓ Always distinguish between macro compilation and execution errors
- ✓ Use %PUT strategically to surface hidden logic and values
- ✓ Understand macro variable scope to avoid unpredictable behavior
- ✓ Quoting issues are subtle when using the correct macro quoting functions
- ✓ Validate inputs early to prevent downstream failures
- ✓ Adopt a disciplined debugging workflow to resolve issues faster and with greater confidence:
  - Isolate the root cause
  - Simplify the code to its essentials
  - Analyze the log for clues and patterns
  - Rebuild incrementally, validating each step

Conclusion

SAS macro debugging does not have to remain a mystery, trial-and-error exercise reserved for experts. What often makes macro issues feel overwhelming is not their complexity, but their invisibility. The fact that macro code generates SAS code behind the scenes, leaving programmers to interpret indirect clues in the log. This Hands-On Training workshop reframes that challenge by making the invisible visible and replacing guesswork with a structured, repeatable approach.

Using the SASHELP.Heart dataset as a consistent and realistic foundation, we explored how macro errors manifest across real analytical tasks, summarization, conditional logic, looping, and parameterized reporting. The examples demonstrated that most macro bugs fall into a handful of categories: unresolved macro variables, scope confusion, flawed logic paths, quoting errors, and invalid inputs. Once recognized, these patterns become far easier to diagnose and correct.

A central takeaway is the critical role of SAS debugging options, MPRINT, MLOGIC, SYMBOLGEN, and related tools, in illuminating macro behavior. These options transform the SAS log into a powerful diagnostic asset, allowing you to trace execution flow, inspect generated code, and confirm how macro variables resolve in real time. When combined with intentional %PUT statements, they provide a level of transparency that turns even cryptic errors into solvable problems.

Equally important is the discipline of defensive programming. By validating parameters, clearly defining variable scope, and anticipating edge cases, you can prevent many common issues before they occur. Debugging, then, becomes a proactive design practice, a skill that improves code reliability, readability, and maintainability.

The systematic workflow emphasized throughout this paper, **isolate, simplify, inspect, test, and refine**, serves as a practical framework you can apply immediately. Rather than rewriting entire macros or making blind changes, this approach encourages incremental testing and evidence-based fixes, saving time and reducing frustration.

Ultimately, effective macro debugging is less about mastering every possible nuance of the SAS macro language and more about developing a calm, methodical mindset supported by the right tools and techniques. With practice, the SAS log becomes less of a barrier and more of a guide, leading you directly to the root cause of issues.

By adopting these strategies, you move from reacting to errors to confidently diagnosing and resolving them, transforming macro debugging from a source of uncertainty into a core professional strength.

Acknowledgments

The author thanks the SEASUG 2026 Conference Committee, particularly the Conference Chairs and Hands-On Workshops (HOW) Section Chair for organizing and supporting a great “in-person” conference event; and SAS Institute Inc. for providing SAS users with wonderful software!

Trademarks

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration. Other brands and product names are trademarks of their respective companies.

About the Author

Kirk Paul Lafler is an internationally recognized AI & Analytics Consultant, Data & Systems Architect, Data Scientist, and Educator who brings deep, cross-disciplinary expertise spanning analytics, data science, SAS, SQL, RDBMS, Python, R, application development, and systems thinking. His work is grounded in the intersection of rigorous technical execution and measurable business outcomes. Recognized for distilling complexity into clear, actionable insight, Kirk equips organizations and professionals to build capability and confidence through hands-on, applied leadership and instruction.

Kirk partners with organizations across industries to improve analytics capabilities, optimize systems, and enable teams to work more efficiently and confidently. As an educator and author, he is passionate about helping professionals grow their skills, strengthen their problem-solving abilities, and achieve better outcomes through smarter use of data and software. As the author of several books, including PROC SQL: Beyond the Basics Using SAS, Third Edition (SAS Press, 2019), Kirk is a frequent invited speaker, educator, and keynote presenter at conferences, symposiums, and professional summits worldwide, and a recipient of 29 “Best” awards for contributed papers, hands-on workshops (HOWs), and posters.

Comments and suggestions are encouraged and can be sent to:

Kirk Paul Lafler, sasNerd
AI & Analytics Consultant, Data & Systems Architect, Data Scientist, Educator, and Author
Specializing in SAS® / Python / R / SQL / RDBMS / AI (GenAI & Agentic AI) / Excel / AWS / Cloud-based Technologies
E-mail: KirkLafler@cs.com
LinkedIn: https://www.linkedin.com/in/KirkPaulLafler/

